

Autoboxing and Unboxing :

As we know, Java is not a pure object oriented language because It accepts primitive data types.

* Except these 8 primitive data types, everything in java is object.

* The basic limitation of these primitive data types, It **cannot** move in the network, Actually only objects are moving in the network.

Example :

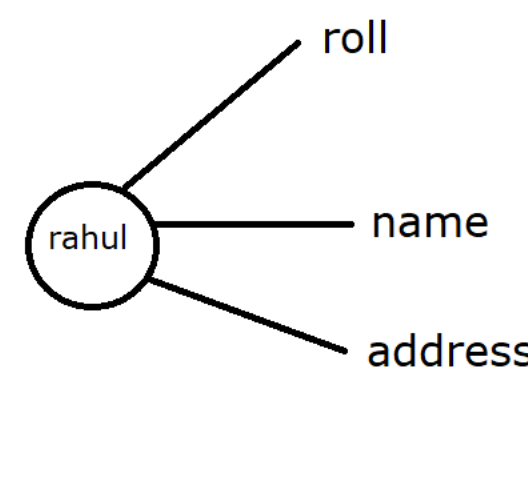
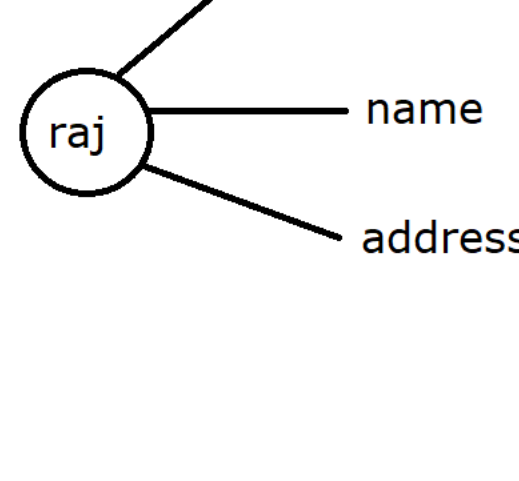
```
public class Student
```

```
{
    int roll;
    String name;
    String address;
```

```
}

Student raj = new Student();
```

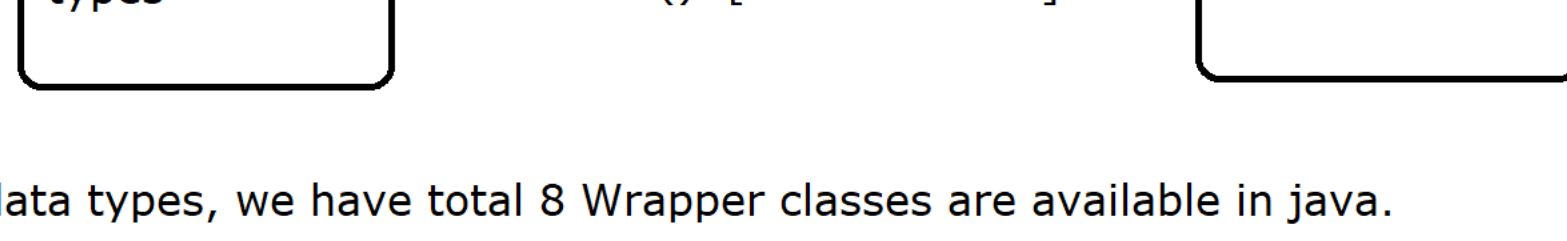
```
Student rahul = new Student();
```



Autoboxing :

* It was introduced from JDK 1.5V onwards.

* It is a technique through which we can convert our primitive data type into corresponding wrapper object.



* For 8 primitive data types, we have total 8 Wrapper classes are available in java.

Primary Data type		Wrapper Object
byte	:	Byte
short	:	Short
int	:	Integer
long	:	Long
float	:	Float
double	:	Double
char	:	Character
boolean	:	Boolean

* Autoboxing facility is available from JDK 1.5V but before Autoboxing, Developer was responsible to convert the primitive data type into Wrapper object manually as shown below :

Example :

```
int x = 10;
Integer obj = Integer.valueOf(x); //Upto JDK 1.4
IO.println(obj);
```

* JDK 1.5V onwards, We can directly assign the primitive data type into corresponding wrapper object but internally java compiler will generate the code as shown below.

```
int x = 10;
Integer obj = x;

↓ Java Compiler generated code

Integer obj = Integer.valueOf(x);
```

Methods :

1) public static Integer valueOf(int x) : It is a predefined method of Integer class which is used to convert the primitive int data type into Integer Wrapper object.

Note : All the 8 Wrapper classes provided valueOf() method to convert from primitive to Wrapper object

2) public static Integer valueOf(String str) : It is a predefined static method of Integer class which is used to convert the String into Integer Wrapper object.

//Programs :

AutoBoxing1.java

```
void main()
{
    int x = 100;
    Integer y = Integer.valueOf(x); //Upto JDK 1.4V
    IO.println(y);
}
```

AutoBoxing2.java

```
void main()
{
    int x = 100;
    Integer y = x; //JDK 1.5V onwards [Autoboxing]
    IO.println(y);
}
```

AutoBoxing3.java

```
void main()
{
    String str = "100";

    //String to primitive int
    int x = Integer.parseInt(str);
    IO.println(x);

    //String to Wrapper object
    Integer obj = Integer.valueOf(str);
    IO.println(obj);
}
```

```
parseInt() : String → int
valueOf() : String → Integer
```

AutoBoxing4.java

```
void main()
{
    byte b = 12;
    Byte b1 = Byte.valueOf(b);
    IO.println(b1);

    short s = 12;
    Short s1 = Short.valueOf(s);
    IO.println(s1);

    int i = 12;
    Integer i1 = Integer.valueOf(i);
    IO.println(i1);

    long x = 90;
    Long x1 = Long.valueOf(x);
    IO.println(x1);

    float f = 12.6F;
    Float f1 = Float.valueOf(f);
    IO.println(f1);

    double d = 12.67;
    Double d1 = Double.valueOf(d);
    IO.println(d1);

    char ch = 'A';
    Character ch1 = Character.valueOf(ch);
    IO.println(ch1);

    boolean val = true;
    Boolean val1 = Boolean.valueOf(val);
    IO.println(val1);
}
```

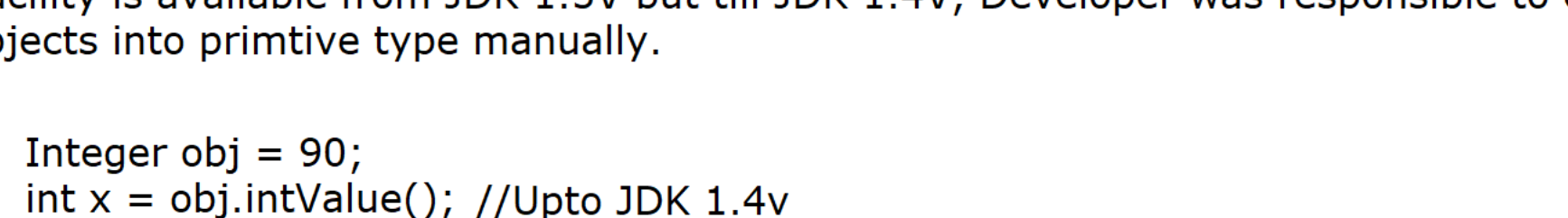
All these 8 Wrapper classes are having valueOf() method

Unboxing :

* It was introduced from JDK 1.5V and it is just opposite of Autoboxing.

* It is used to convert the Wrapper object into corresponding primitive type.

Wrapper Object		Primitive type
Byte	-	byte
Short	-	short
Integer	-	int
Long	-	long
Float	-	float
Double	-	double
Character	-	char
Boolean	-	boolean



* Unboxing facility is available from JDK 1.5V but till JDK 1.4V, Developer was responsible to convert the Wrapper objects into primitive type manually.

Example :

```
Integer obj = 90;
int x = obj.intValue(); //Upto JDK 1.4v
IO.println(x);
```

* JDK 1.5V onwards, We can directly assign a wrapper object into primitive type as a part of Unboxing concept.

Example :

```
Integer obj = 90;
int x = obj; //JDK 1.5V

↓ Compiler generated code

int x = obj.intValue();
```

Method :

public int intValue() : It is a non static method of Integer class which is used to convert the Integer Wrapper object into primitive int.

Note : All these 8 wrapper classes provided xxxValue() method to convert the Wrapper object into corresponding primitive type

Unboxing1.java

```
void main()
{
    Integer obj = 100;
    int x = obj.intValue(); //Till JDK 1.4V

    Character ch = 'A';
    char c1 = ch.charValue(); //Till JDK 1.4V

    Boolean b1 = false;
    boolean b = b1.booleanValue(); //Till JDK 1.4V

    IO.println(x+" : "+c1+" : "+b);
}
```

Unboxing2.java

```
void main()
{
    Integer obj = 100;
    int x = obj; //JDK 1.5V

    Character ch = 'A';
    char c1 = ch; //JDK 1.5V

    Boolean b1 = false;
    boolean b = b1; //JDK 1.5V

    IO.println(x+" : "+c1+" : "+b);
}
```

* Type Conversion (One data type to another data type) is possible with **primitive data types** but the same is not possible with Wrapper classes as shown below.

```
Long x = 12; //Invalid
```

```
Float f = 10; //Invalid
```

```
Double d = 12; //Invalid
```

UnboxingDemo3.java

```
void main()
{
    Long x = 1L;
    IO.println(x);

    Float f = 10F;
    IO.println(f);

    Double d = 123.0;
    Double d1 = 15D;
    IO.println(d+" : "+d1);
}
```

Method Overloading Resolution :

* While working with method overloading, java compiler will select the method (Static Polymorphism OR Early binding) based on the method parameter.

* While selecting a method based on the parameter, Java compiler provides the priority using following rules :

1) Exact match rule :

If java compiler will find the exact match then It will execute that particular method.

2) Most Specific Type :

Here compiler will give the priority to the most specific data type

```
double > float    [float is the more specific type]
float > long      [long is the more specific type]
long > int
int > char
int > short      Note : There is no relation b/w short & char
short > byte
```

3) While working with widening, Autoboxing and Var args, Java compiler provides the priority on the following basis :

Widening -> Autoboxing -> Var Args **[WAV]**